
Primitive Data Types (Fixed Size in Java)

Data Type	Size (bits)	Size (bytes)	Notes
`boolean`	JVM-dependent	1 byte	Even though a boolean is just **1 bit**, it occupies **1 byte** for memory alignment.
`char`	16 bits	2 bytes	Uses **UTF-16** encoding.
`int`	32 bits	4 bytes	Standard integer type in Java.
`float`	32 bits	4 bytes	IEEE 754 single-precision floating-point.
`double`	64 bits	8 bytes	IEEE 754 double-precision floating-point.

> **Note:** Java uses a 64-bit **heap memory model** (Compressed OOP) where object references typically take **4 bytes** (instead of 8 bytes) if the heap size is below 32GB.

Collection Types (Memory Usage: Empty Constructor vs Initial Size)

Collections are **objects**, so their size depends on internal structures.

`ArrayList<E>` (Backed by an array)

- `new ArrayList<>()` → **Default capacity** = `10` (but starts with an empty array)
- `new ArrayList<>(0)` → Internal array = **size 0**.
- `new ArrayList<>(1000)` → Internal array = **size 1000**.

📌 **Memory Overhead:**

- An empty `ArrayList` object itself **≈ 24 bytes** (without elements).
- The internal `Object[]` **starts empty** and grows dynamically.

`LinkedList<E>` (Doubly Linked List)

- `new LinkedList<>()` → Always **empty** (no array, only head/tail references).
- Each element **Node** stores:
    - **Data (Reference) → 4 bytes**
    - **Next Pointer → 4 bytes**
    - **Prev Pointer → 4 bytes**
    - **Object Header → 12 bytes**

      Total per element ≈ 24 bytes.

`HashMap<K, V>` (Backed by an array of Nodes)

- `new HashMap<>()` → Default **capacity = 16** (with load factor **0.75**).
- `new HashMap<>(32)` → Table size = **32** (rounded to nearest power of 2).
- **Memory Usage for Each Entry**:
    - **HashMap object itself**: **≈ 32 bytes**
    - **Each Entry (Node<K, V>)**: **≈ 32 bytes** (key, value, next pointer, hash)
    - **Empty HashMap with default capacity**: **≈ 144 bytes** (empty table array + object overhead).

`HashSet<E>`

- Internally uses a **HashMap<E, Object>** where the value is a dummy object.
- Similar memory usage as `HashMap`.

Summary of Collection Memory Usage

Collection	Empty Constructor	Initial Size Constructor (e.g., 1000)
`ArrayList<E>`	**≈ 24 bytes** (no array initially)	Internal array of **1000 elements** (≈ 4000 bytes for Object references)
`LinkedList<E>`	**≈ 24 bytes** (only head/tail refs)	Each node is **24 bytes** (so 1000 elements → **24 KB**)
`HashMap<K,V>`	**≈ 144 bytes** (empty table)	Hash table pre-allocated (`capacity 1024` ≈ 32 KB)
`HashSet<E>`	Same as `HashMap<E, Object>`	Similar to `HashMap`

JVM memory optimization involves understanding **heap memory management, garbage collection (GC), and object sizing**. Let's break it down:

JVM Memory Structure

The JVM allocates memory mainly in **two regions**:

1. **Heap Memory** (used for objects)
    - **Young Generation (Eden, Survivor)**
    - **Old Generation (Tenured)**
    - **Metaspace (Class metadata, replaces PermGen)**
2. **Stack Memory** (used for method calls, local variables)

Optimization Techniques

1️⃣ Reduce Object Overhead (Efficient Data Structures)

✅ Prefer `ArrayList` over `LinkedList`

- `LinkedList` uses **24 bytes per node** (extra pointers).
- `ArrayList` has **less overhead** (just an array of references).

✅ Use `HashMap` with proper initial size

- Avoid resizing costs by setting `new HashMap<>(expectedSize)`.
- Default `load factor = 0.75`, so `capacity ≈ expectedSize / 0.75`.

✅ Use `primitive arrays` instead of `Collections` (where possible)

- `int[]` is **more memory-efficient** than `ArrayList<Integer>` (which stores boxed `Integer` objects).

2️⃣ Reduce Object Creation (Object Reuse)

✅ Use `StringBuilder` instead of `String` (for concatenation)

- `String` is immutable → Every modification creates **a new object**.
- `StringBuilder` avoids unnecessary allocations.

✅ Use `ThreadLocal` or Object Pools

- If creating many short-lived objects, `ThreadLocal` can **cache instances**.
- **Example:** Instead of `new SimpleDateFormat()`, use `ThreadLocal<SimpleDateFormat>`.

3️⃣ Optimize JVM Memory Settings (Heap Tuning)

✅ Set Heap Size Properly (`-Xms` and `-Xmx`)

- Example: `-Xms2g -Xmx2g` (Set min/max heap to avoid resizing overhead).
- Use `-XX:+UseG1GC` for balanced GC performance.

✅ Enable Escape Analysis (`-XX:+DoEscapeAnalysis`)

- JVM **eliminates** unnecessary heap allocations → objects stay on stack.

4️⃣ Minimize Garbage Collection (GC) Pressure

- **Avoid unnecessary object creation** (e.g., prefer `int` over `Integer`).
- Use **long-lived objects** for frequently used data (`static` caches, object pools).
- **Profile GC performance** using tools like:
  - `jvisualvm`
  - `jconsole`
  - `-XX:+PrintGCDetails`

🚀 Key Takeaways

✔ **Use memory-efficient data structures** (ArrayList, primitive arrays).
✔ **Minimize object creation** (reuse objects, use `StringBuilder`).
✔ **Tune JVM heap settings** (`-Xms`, `-Xmx`, `G1GC`).
✔ **Reduce GC overhead** (escape analysis, pooling).